

CONCEPTION PHASE: Airbnb Database System

Course: Build a Data Mart in SQL (DLBDSPBDM01)

Name: Ipadeola Oluwatoyin Eniola

Matriculation Number: 92130758

Date: 26-12-2025

1. PROBLEM SUMMARY

Modern property rental platforms such as Airbnb manage large amounts of data from various stakeholders, including hosts, guests, property listings, bookings, payments, and reviews. Without a properly organized relational database, this information can become scattered across different systems, causing several major problems:

Data Inconsistency and Redundancy: Property information, user details, and booking records are often duplicated or stored in incompatible formats, resulting in significant discrepancies in availability calendars, pricing, and user profiles that must be rectified.

Operational Inefficiencies: The lack of integrated data relationships severely hampers our ability to track booking histories, payment statuses, and communication between hosts and guests. This inefficiency leads to delayed reservation confirmations, payment processing errors, and unnecessary challenges in dispute resolution that we can no longer tolerate.

Limited Analytical Capabilities: Without a centralized database, extracting valuable insights on booking trends, host performance, revenue analytics, and customer behavior is not only cumbersome but also error-prone. We need a system that allows for accurate and efficient data analysis.

Scalability Constraints: As user numbers and transaction volumes rise, current ad-hoc data storage methods will fail to support the increasing load, resulting in system slowdowns and the unacceptable risk of data loss.

This project will implement a comprehensive relational database system specifically designed for Airbnb-like platforms to effectively address these challenges. The solution will normalize data storage, enforce referential integrity through primary and foreign keys, and establish clear relationships among all entities. This structured approach will guarantee data accuracy, enable efficient querying, and provide a robust foundation for future platform enhancements and detailed analytical reporting. We must take these steps to ensure our platform's success.

2. REQUIREMENTS SPECIFICATION

2.1 User Roles and Groups

The system is designed to support four primary user roles, each with specific responsibilities and data requirements:

- **Guest:** Guests actively search for, book, and stay at listed properties. They take charge of browsing, making bookings, processing payments, providing reviews, and communicating effectively with hosts.
- **Host:** Hosts are responsible for listing and managing their properties. They oversee property listings, maintain availability, approve bookings, and ensure timely payment receipt.

- **Administrator:** Administrators maintain platform integrity by overseeing system operations, managing user accounts, resolving disputes, and enforcing compliance with platform policies.
- **Payment Processor:** This automated role handles all financial transactions, processes payments, manages refunds, and meticulously maintains transaction audit trails.

2.2. Actions Performed by Each Role

Guest Actions:

- Register and establish a user profile
- Search and filter properties by location, price, dates, and amenities with precision
- View detailed property information, including photos, reviews, and host statistics
- Make seamless booking reservations for selected properties
- Process payments through secure payment gateways without hassle
- Communicate directly with hosts through a robust messaging system
- Submit honest reviews and ratings immediately after their stay
- Manage personal booking history and preferences effectively

Host Actions:

- Register as a host and promptly complete verification
- Create and manage property listings with detailed descriptions, high-quality photos, and competitive pricing
- Set and update property availability calendars to maximize rental opportunities
- Establish booking rules, including minimum stay requirements and cancellation policies
- Approve or decline booking requests decisively
- Manage communications with both potential and current guests professionally
- Actively view and respond to guest reviews to enhance service
- Track earnings consistently and ensure timely payouts
- Generate insightful performance reports to guide future actions

Administrator Actions:

- Manage user accounts rigorously, including approval, suspension, and deletion
- Monitor and moderate property listings to guarantee compliance
- Address and resolve disputes between guests and hosts promptly
- Review system audit logs and activity reports regularly
- Configure platform settings and enforce policies effectively
- Manage promotional campaigns and discounts strategically
- Generate comprehensive platform-wide analytics and reports

Payment Processor Actions:

- Process booking payments from guests
- Handle refunds and cancellations
- Manage payout distributions to hosts
- Maintain transaction audit trails
- Interface with external payment gateways
- Generate financial reconciliation reports

2.3 Required Data and Functions

1. Core Data Requirements:

User Data:

- Ensure personal information is collected, including name, email, phone number, and profile picture.
- Assign roles and permissions effectively to manage user access.
- Maintain robust authentication credentials to safeguard user accounts.
- Collect links to social media along with verification status for enhanced user engagement.
- Track account creation dates and activity logs for comprehensive user insights.

Property Data:

- Gather complete property details, including title, description, type, and size.
- Record precise location information, including address, city, country, and coordinates.
- Establish clear pricing information, covering base rates, seasonal pricing, and applicable fees.
- Highlight amenities and features that differentiate properties.
- Incorporate high-quality photos and media files to attract potential guests.
- Maintain an up-to-date availability calendar to prevent double bookings.
- Clearly outline house rules and policies for guest compliance.

Booking Data:

- Booking reference and status details
- Comprehensive guest and property information
- Clear check-in and check-out dates, along with the total duration of the stay
- A detailed pricing breakdown, encompassing nightly rates, cleaning fees, and taxes
- An up-to-date payment status and complete history
- All special requests and requirements to ensure a smooth experience

Payment Data:

- Record transaction IDs and timestamps for all financial activities.
- Accurately capture amounts and currency involved in transactions.
- Detail payment method information for transparency.
- Monitor payment status meticulously (pending, completed, refunded, failed).
- Maintain comprehensive payout records for hosts.
- Establish clear financial audit trails for accountability.

Review Data:

- Implement a structured rating system that covers overall performance, cleanliness, accuracy, communication, etc.
- Encourage feedback through text reviews and comments.
- Ensure transparency by documenting reviewer and reviewee information.
- Facilitate responses from the reviewed party to foster communication.
- Track review dates and timestamps to maintain context.

Support Data:

- Maintain communication logs between users to enhance the support experience.
- Document dispute records and resolutions for accountability.
- Track support tickets and their status to ensure timely resolutions.
- Regularly review system audit logs to monitor platform integrity.
- Distribute platform notifications effectively to keep users informed.

2.4. System Functions Required:

- 1. User Management:** Implement comprehensive registration, authentication, profile management, and role assignments.
- 2. Property Catalog:** Streamline listing creation/editing, enable thorough search and filtering, and manage availability efficiently.
- 3. Booking Engine:** Facilitate seamless reservation processing, conduct real-time availability checks, and ensure accurate pricing calculations.
- 4. Payment Processing:** Guarantee secure transactions, handle refunds proficiently, and distribute payouts efficiently.
- 5. Communication System:** Enable user messaging and ensure effective notification delivery.
- 6. Review System:** Support rating submissions, display reviews prominently, and manage responses effectively.

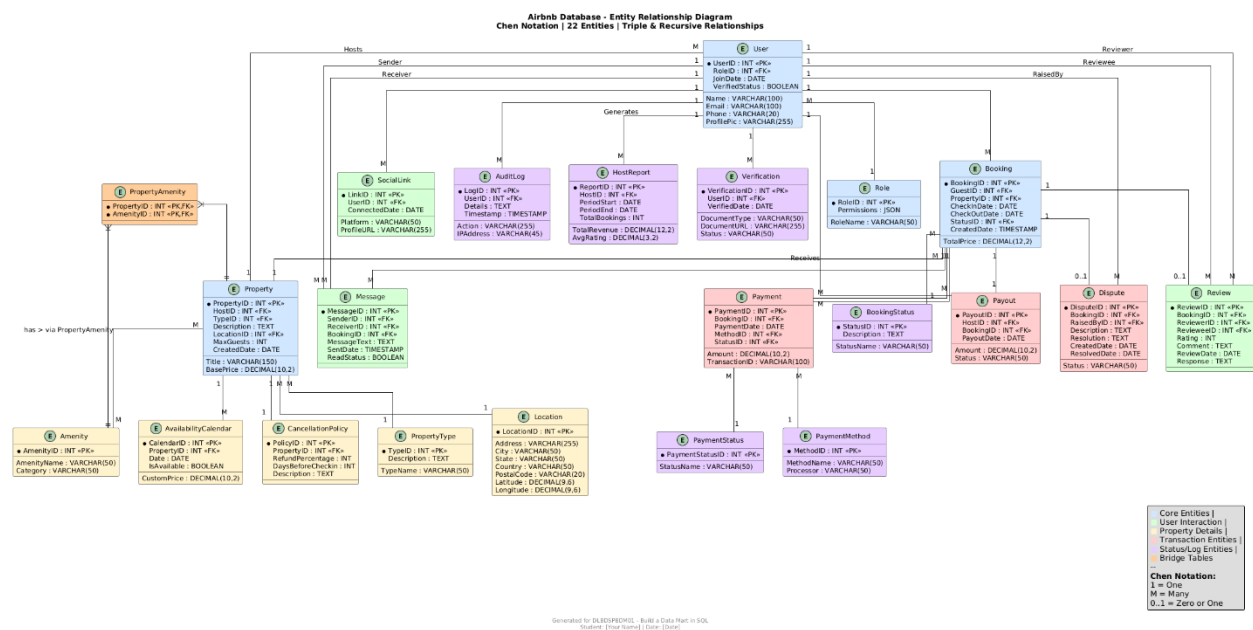
7. Administrative Tools: Equip administrative personnel with robust user moderation, dispute resolution capabilities, and comprehensive reporting dashboards.

8. Analytical Functions: Provide detailed revenue reporting, conduct thorough occupancy analysis, and track key performance metrics.

9. Security Features: Implement data encryption, establish stringent access controls, and maintain comprehensive audit logging to protect user data

3. ENTITY RELATIONSHIP MODEL (ERM)

3.1 ER Diagram Overview



This ER model is composed of 22 distinct entities that effectively capture every aspect of the Airbnb platform. It employs Chen notation for clear representation of cardinality and incorporates both triple relationships and recursive relationships as needed, ensuring a comprehensive representation of the system.

3.2 Entity List with Key Attributes

- User** (UserID [PK], RoleID [FK], Name, Email, Phone, ProfilePic, JoinDate, VerifiedStatus)
- Role** (RoleID [PK], RoleName, Permissions)
- Property** (PropertyID [PK], HostID [FK], TypeID [FK], Title, Description, LocationID [FK], BasePrice, MaxGuests, CreatedDate)
- PropertyType** (TypeID [PK], TypeName, Description)
- Amenity** (AmenityID [PK], AmenityName, Category)
- PropertyAmenity** (PropertyID [PK,FK], AmenityID [PK,FK]) *[Resolves M:N relationship]*
- Location** (LocationID [PK], Address, City, State, Country, PostalCode, Latitude, Longitude)

- **Booking** (BookingID [PK], GuestID [FK], PropertyID [FK], CheckInDate, CheckOutDate, TotalPrice, StatusID [FK], CreatedDate)
- **BookingStatus** (StatusID [PK], StatusName, Description)
- **Payment** (PaymentID [PK], BookingID [FK], Amount, PaymentDate, MethodID [FK], StatusID [FK], TransactionID)
- **PaymentMethod** (MethodID [PK], MethodName, Processor)
- **PaymentStatus** (PaymentStatusID [PK], StatusName)
- **Payout** (PayoutID [PK], HostID [FK], BookingID [FK], Amount, PayoutDate, Status)
- **Review** (ReviewID [PK], BookingID [FK], ReviewerID [FK], RevieweeID [FK], Rating, Comment, ReviewDate, Response) *[Recursive]*
- **AvailabilityCalendar** (CalendarID [PK], PropertyID [FK], Date, IsAvailable, CustomPrice)
- **Message** (MessageID [PK], SenderID [FK], ReceiverID [FK], BookingID [FK], MessageText, SentDate, ReadStatus) *[Recursive]*
- **Dispute** (DisputeID [PK], BookingID [FK], RaisedByID [FK], Description, Status, Resolution, CreatedDate, ResolvedDate)
- **AuditLog** (LogID [PK], UserID [FK], Action, Details, Timestamp, IPAddress)
- **SocialLink** (LinkID [PK], UserID [FK], Platform, ProfileURL, ConnectedDate)
- **HostReport** (ReportID [PK], HostID [FK], PeriodStart, PeriodEnd, TotalBookings, TotalRevenue, AvgRating)
- **CancellationPolicy** (PolicyID [PK], PropertyID [FK], RefundPercentage, DaysBeforeCheckin, Description)
- **Verification** (VerificationID [PK], UserID [FK], DocumentType, DocumentURL, Status, VerifiedDate)

3.3 Triple Relationships

1. Booking Triple Relationship:

- Connects: **Guest (User) ↔ Booking ↔ Property**
- Description: A Guest creates a Booking for a specific Property. The Booking entity serves as the connecting table between User (as Guest) and Property.
- Cardinality: User(1) — (M)Booking — (1)Property

2. Payment Triple Relationship:

- Connects: **Booking ↔ Payment ↔ PaymentMethod**
- Description: Each Payment is associated with one Booking and uses one PaymentMethod. This ensures traceability from booking to payment method.
- Cardinality: Booking(1) — (M)Payment — (1)PaymentMethod

3. Review Triple Relationship:

- Connects: **Booking** ↔ **Review** ↔ **User (as Reviewer)** ↔ **User (as Reviewee)**
- Description: A Review is linked to a Booking, with one User reviewing another User. This creates a triple relationship through the Review entity.
- Cardinality: Booking(1) — (M)Review — (1)User (as Reviewer) — (1)User (as Reviewee)

3.4 Recursive Relationships (2 implemented)

1. Message Recursive Relationship:

- Relationship: **User** → **Message** → **User**
- Description: A User (Sender) sends a Message to another User (Receiver). Both Sender and Receiver reference the User entity.
- Implementation: Message table has SenderID [FK → User] and ReceiverID [FK → User]

2. Review Recursive Relationship:

- Relationship: **User** → **Review** → **User**
- Description: A User (Reviewer) writes a Review about another User (Reviewee). Both references point to the User entity.
- Implementation: Review table has ReviewerID [FK → User] and RevieweeID [FK → User]

3.5 Cardinality Specifications (Chen Notation)

- User (1) — (M) Booking (Each user can have many bookings, each booking belongs to one user)
- Property (1) — (M) Booking (Each property can have many bookings, each booking is for one property)
- User (1) — (M) Property (Each host can list many properties, each property belongs to one host)
- Property (M) — (M) Amenity (Each property can have many amenities, each amenity can be in many properties)
- Booking (1) — (0..1) Review (Each booking can have zero or one review)
- User (1) — (M) Message [as Sender] (Each user can send many messages)
- User (1) — (M) Message [as Receiver] (Each user can receive many messages)
- Booking (1) — (1..M) Payment (Each booking has at least one payment, can have multiple for deposits/final payments)

4. DATA DICTIONARY

Entity	Attribute	Data Type	Description	PK/FK	Constraints
User	UserID	INT	Unique identifier for each user	PK	Auto-increment
	RoleID	INT	Reference to user's role	FK → Role	NOT NULL
	Name	VARCHAR(100)	Full name of user		NOT NULL
	Email	VARCHAR(100)	User's email address		UNIQUE, NOT NULL
	Phone	VARCHAR(20)	Contact phone number		
	ProfilePic	VARCHAR(255)	URL to profile picture		
	JoinDate	DATE	Date user joined platform		DEFAULT CURRENT_DATE
	VerifiedStatus	BOOLEAN	Account verification status		DEFAULT FALSE
Role	RoleID	INT	Unique role identifier	PK	Auto-increment
	RoleName	VARCHAR(50)	Name of role (Guest, Host, Admin)		NOT NULL
	Permissions	JSON	JSON of role permissions		
Property	PropertyID	INT	Unique property identifier	PK	Auto-increment

Entity	Attribute	Data Type	Description	PK/FK	Constraints
	HostID	INT	Property owner reference	FK → User	NOT NULL
	TypeID	INT	Property type reference	FK → PropertyType	NOT NULL
	Title	VARCHAR(150)	Property listing title		NOT NULL
	Description	TEXT	Detailed property description		
	LocationID	INT	Property location reference	FK → Location	NOT NULL
	BasePrice	DECIMAL(10,2)	Standard price per night		NOT NULL
	MaxGuests	INT	Maximum occupancy		NOT NULL
	CreatedDate	DATE	Listing creation date		DEFAULT CURRENT_DATE
Booking	BookingID	INT	Unique booking identifier	PK	Auto-increment
	GuestID	INT	Guest who made booking	FK → User	NOT NULL
	PropertyID	INT	Booked property	FK → Property	NOT NULL
	CheckInDate	DATE	Start date of stay		NOT NULL
	CheckOutDate	DATE	End date of stay		NOT NULL

Entity	Attribute	Data Type	Description	PK/FK	Constraints
	TotalPrice	DECIMAL(12,2)	Calculated total cost		NOT NULL
	StatusID	INT	Current booking status	FK → BookingStatus	NOT NULL
	CreatedDate	TIMESTAMP	Booking creation timestamp		DEFAULT CURRENT_TIME STAMP
Payment	PaymentID	INT	Unique payment identifier	PK	Auto-increment
	BookingID	INT	Associated booking	FK → Booking	NOT NULL
	Amount	DECIMAL(10,2)	Payment amount		NOT NULL
	PaymentDate	DATE	Date of payment		NOT NULL
	MethodID	INT	Payment method used	FK → PaymentMethod	NOT NULL
	StatusID	INT	Payment status	FK → PaymentStatus	NOT NULL
	TransactionID	VARCHAR(100)	External transaction reference		
Review	ReviewID	INT	Unique review identifier	PK	Auto-increment
	BookingID	INT	Associated booking	FK → Booking	NOT NULL
	ReviewerID	INT	User writing review	FK → User	NOT NULL

Entity	Attribute	Data Type	Description	PK/FK	Constraints
	RevieweeID	INT	User being reviewed	FK → User	NOT NULL
	Rating	INT	Numerical rating (1-5)		CHECK (1-5)
	Comment	TEXT	Review text content		
	ReviewDate	DATE	Date review submitted		DEFAULT CURRENT_DATE
	Response	TEXT	Host response to review		
Message	MessageID	INT	Unique message identifier	PK	Auto-increment
	SenderID	INT	User sending message	FK → User	NOT NULL
	ReceiverID	INT	User receiving message	FK → User	NOT NULL
	BookingID	INT	Associated booking (optional)	FK → Booking	
	MessageText	TEXT	Message content		NOT NULL
	SentDate	TIMESTAMP	Message timestamp		DEFAULT CURRENT_TIMESTAMP
	ReadStatus	BOOLEAN	Whether message was read		DEFAULT FALSE

5. WORK SUMMARY

This conception phase involved systematic analysis and design of a database solution for an Airbnb-style platform. The process followed these key steps:

1. **Problem Analysis:** Identified pain points in current property rental platforms including data fragmentation, operational inefficiencies, and scalability issues.
2. **Requirements Gathering:** Defined four user roles (Guest, Host, Admin, Payment Processor) with detailed actions and data requirements for each.
3. **Entity Identification:** Developed a comprehensive list of 22 entities covering all aspects of the platform from user management to financial transactions.
4. **Relationship Modeling:** Designed both standard and complex relationships including:
 - Three triple relationships (Booking-Guest-Property, Payment-Booking-Method, Review-Booking-User)
 - Two recursive relationships (Message between Users, Review between Users)
 - Proper cardinality specifications using Chen notation
5. **Data Dictionary Creation:** Detailed specification of all attributes, outlining appropriate data types, constraints, and descriptions to ensure clarity for implementation.

The resulting Entity-Relationship (ER) model delivers a normalized and scalable foundation, supporting all essential platform functionalities while ensuring data integrity and accommodating future analytical needs. This design phase establishes a definitive blueprint for the subsequent development and implementation phases.